Kurs Django — Lekcja 4

Modele i ORM (podstawy)

Cel lekcji

Student potrafi zdefiniować model, wykonać migrację i wykonywać podstawowe zapytania ORM.

Blok 1: Teoria (10–15 min)

Czym jest ORM

Object-Relational Mapping — piszemy klasy Pythona, a Django samo generuje odpowiednie zapytania SQL. Nie musimy pisać SQL-a ręcznie (choć można, gdy trzeba).

Migracje — dlaczego dwa kroki

- makemigrations — Django patrzy na modele i tworzy plik z opisem zmian (nie dotyka jeszcze bazy)
- migrate — dopiero to polecenie faktycznie wykonuje zmiany w bazie danych

To rozdzielenie jest celowe: plik migracji można przejrzeć, dodać do repozytorium (git), cofnąć.

Blok 2: Ćwiczenia w terminalu

Krok 1 — definicja modelu

W blog/models.py:

```
from django.db import models

class Post(models.Model):
    tytul = models.CharField(max_length=200)
    tresc = models.TextField()
    data_publikacji = models.DateTimeField(auto_now_add=True)
    opublikowany = models.BooleanField(default=True)

    def __str__(self):
        return self.tytul
```

Omówić typy pól: CharField (krótki tekst, wymaga max_length), TextField (długi tekst), DateTimeField, BooleanField, IntegerField. Metoda __str__ decyduje, jak obiekt wygląda w panelu admina i w konsoli — warto ją zawsze dodawać.

Krok 2 — migracje (rutyna dwuetapowa)

```
python manage.py makemigrations
```

Sprawdzić co powstało:

```
python manage.py sqlmigrate blog 0001
```
(pokazuje wygenerowany SQL — dobre ćwiczenie, żeby zobaczyć „pod maską”)

Zastosowanie migracji:

```
python manage.py migrate
```

Powtórzyć zasadę z Lekcji 1: każda zmiana w models.py → makemigrations → migrate. Bez drugiego kroku zmiany nie trafią do bazy.

Krok 3 — Django shell

```
python manage.py shell
```
W konsoli interaktywnej:

```
from blog.models import Post

# tworzenie obiektu
p = Post.objects.create(tytul="Pierwszy post", tresc="To jest treść posta.")
print(p)

# pobieranie wszystkich
Post.objects.all()

# filtrowanie
Post.objects.filter(opublikowany=True)

# pobranie jednego konkretnego obiektu
Post.objects.get(id=1)
```

Krok 4 — aktualizacja i usuwanie w shellu

```
post = Post.objects.get(id=1)
post.tytul = "Zmieniony tytuł"
post.save()

post.delete()
```

Krok 5 — dodanie kilku rekordów testowych

```
Post.objects.create(tytul="Drugi post", tresc="Treść drugiego posta.")
Post.objects.create(tytul="Trzeci post", tresc="Treść trzeciego posta.", opublikowany=False)

Post.objects.count()
Post.objects.filter(opublikowany=False)
```
Wyjście z shella: exit()

Blok 3: Zadanie dla studentów

- Dodać do modelu Post nowe pole autor = models.CharField(max_length=100, default="Anonim")
- Wykonać makemigrations + migrate — zaobserwować, że Django pyta o wartość domyślną dla istniejących rekordów, jeśli pole nie ma default (dobra okazja, żeby to pokazać na żywo)

- W shellu: policzyć ile jest postów opublikowanych a ile nie, korzystając z .filter().count()

Checklist nawyków

☐ Zmiana w models.py → makemigrations

☐ Zawsze po makemigrations → migrate

☐ Każdy model ma metodę __str__